

Component, State & ConditionalRendering

1. Components

There are two ways to write React Components.

They are:

- Functional Components
- Class Components

1.1 Functional Components

These are JavaScript functions that take props as a parameter if necessary and return react element (JSX).

```
const Welcome = () => <h1>Hello, User</h1>;
```

```
export default Welcome;
```

1.2 Class Components

These components are built using an ES6 class.

To define a React Class Component,

- Create an ES6 class that extends `React.Component`.
- Add a single empty method to it called `render()`.

1.2.1 extends

The `extends` keyword is used to inherit methods and properties from the `React.Component`.

1.2.2 render()

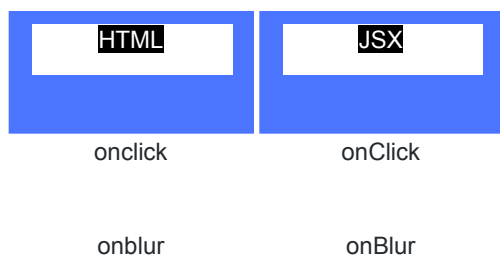
The `render()` method is the only required method in a class component. It returns the JSX element

2. React Events

Handling events with React elements is very similar to handling events on DOM elements. There are some syntax differences:

1. React events are named using **camelCase**, rather than **lowercase**.

Example:



onChange

onChange

2. With JSX, you pass a **function** as the event handler rather than a **string**.

Example: <button onClick="activateLasers()">Activate Lasers</button>

<button onClick={activateLasers}>Activate Lasers</button>

We should not call the function when we add an event in JSX.

```
class MyComponent extends Component {  
  
  handleClick = () => {  
  
    console.log("clicked")  
  
  }  
  
  render() {  
  
    return <button onClick={this.handleClick()}>Click Me</button>  
  
  }  
}
```

In the above function, the handleClick is called instead of passed as a reference.

```
class MyComponent extends Component {  
  
  handleClick = () => {  
  
    console.log("clicked")  
  
  }  
}
```

```
render() {  
  
  return <button onClick={this.handleClick}>Click Me</button>  
  
}  
  
}
```

In the above function, the

handleClick is passed as a reference. So, the function is not being called every time the component renders.

Providing Arrow Functions

To not change the context of this, we have to pass an arrow function to the event.

```
class MyComponent extends Component {  
  
  handleClick() {  
  
    console.log(this) // undefined  
  
  }  
  
  render() {  
  
    return <button onClick={this.handleClick}>Click Me</button>  
  
  }  
  
}  
  
class MyComponent extends Component {  
  
  handleClick = () => {
```

```
    console.log(this) // MyComponent {...}

  }

  render() {

    return <button onClick={this.handleClick}>Click Me</button>

  }

}
```

3. State

The state is a JS object in which we store the component's data that changes over time.

When the state object changes, the component re-renders.

Intialising State:

```
class Counter extends Component {

  state = { count: 0 }

  render() {

    const { count } = this.state;

    return <p className="count">{count}</p>;

  }

}
```

3.1 Updating State

We can update the state by using

`setState()`. We can provide function/object as an argument to set the state.

Providing Function as an Argument:

Syntax:

```
this.setState( prevState => ({... }) )
```

Here the previous state is sent as a parameter to the callback function.

```
onIncrement = () => {  
  
  this.setState((prevState) =>  
  
    console.log(`previous state value ${prevState.count}`)  
  
  )  
  
}
```

3.2 State Updates are Merged

State updates are merged. It means that when you update only one key-value pair in the state object, it will not affect the other key-value pairs in the state object.

// For example let's say your state is as followed:

```
state = { key1 : value1, key2 : value2 }
```

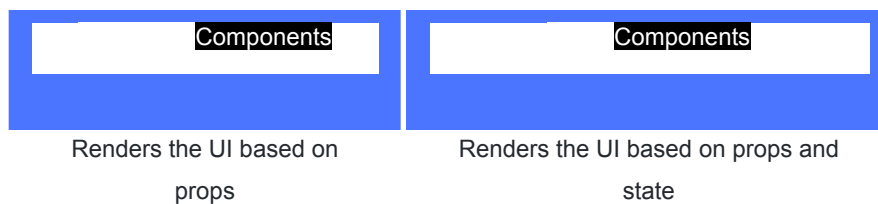
// if you use `this.setState` such as :

```
this.setState((prevState) => ({ prevState.key1 : value3 })))
```

// your new state will be :

```
state = { key1 : value3, key2 : value2 }
```

3.3 Functional Components vs Class Components



Use Class Components whenever the state is required. Otherwise, use the Functional components.

4 . Conditional Rendering

Conditional Rendering allows us to render different elements or components based on a condition.

Different ways to implement **Conditional Rendering** are:

- Using an If...Else Statement
- Using Element Variables
- Using Ternary Operators
- Using Logical && Operator

4.1 Using an If...Else Statement

```
import { Component } from "react"
```

```
import './App.css'
```

```
class App extends Component {
```

```
  state = { isLoggedIn: true }
```

```
  renderAuthButton = () => {
```

```
    const {isLoggedIn} = this.state
```

```
    if (isLoggedIn === true) {
```

```
      return <button>Logout</button>
```

```
    }
```

```
    return <button>Login</button>
```

```
  }
```

```
  render() {
```

```
    return (
```

```
      <div className="container">
```



```
        {this.renderAuthButton()}  
  
      </div>  
  
    )  
  
  }  
  
}
```

```
export default App
```

4.2 Using Element Variables

```
import { Component } from "react"
```

```
import './App.css'
```

```
class App extends Component {
```

```
  state = { isLoggedIn: true }
```

```
  render() {
```

```
    const { isLoggedIn } = this.state
```

```
    let authButton
```

```
    if (isLoggedIn) {
```

```
    authButton = <button>Logout</button>

  } else {

    authButton = <button>Login</button>

  }

  return (

    <div className="container">

      <h1>React JS</h1>

      {authButton}

    </div>

  )

}

}
```

export default App

4.3 Using Ternary Operators

```
import { Component } from "react"

import './App.css'
```

```
class App extends Component {  
  
  render() {  
  
    const { isLoggedIn } = this.state  
  
    return (  
  
      <div className="container">  
  
        {isLoggedIn ? <button>Logout</button> : <button>Login</button>}  
  
      </div>  
  
    )  
  
  }  
  
}
```

```
export default App
```

4.4 Using Logical && Operator

```
import { Component } from "react"
```

```
import './App.css'
```

```
class App extends Component {
```

```
render() {  
  
  const { isLoggedIn } = this.state  
  
  return (  
  
    <div className="container">  
  
      {isLoggedIn && <button>Logout</button>}  
  
      {!isLoggedIn && <button>Login</button>}  
  
    </div>  
  
  )  
  
}
```

```
export default App
```

Conditional Rendering can be achieved using inline styles or adding classes with CSS display property with value none. However, it is not preferable.

5. Sending Function as Callback

We can pass functions as props to child components.

Syntax: <ComponentName functionName={this.functionName} />

6. Input Element

In React, the Input Element

value can be handled in two ways:

- Controlled Input
- Uncontrolled Input

6.1 Controlled Input

If the Input Element

value is handled by a React State then it is called **Controlled Input**. Controlled Inputs are the React Suggested way to handle Input Element value.

Example:

```
import {Component} from 'react'
```

```
class App extends Component {
```

```
  state = {
```

```
    searchInput: "",
```

```
  }
```

```
  onChangeSearchInput = event => {
```

```
this.setState({
```

```
  searchInput: event.target.value,
```

```
  })
```

```
}
```

```
render() {
```

```
  const {searchInput} = this.state
```

```
  return (
```

```
    <input
```

```
      type="text"
```

```
      onChange={this.onChangeSearchInput}
```

```
      value={searchInput}
```

```
    />
```

```
  )
```

```
}
```

```
}
```

```
export default App
```

6.2 Uncontrolled Input

If the Input Element

value is handled by the browser itself then it is called **Uncontrolled Input**.

Uncontrolled inputs are like traditional HTML form inputs. Its

value can only be set by a user, but not programmatically. However, in controlled input value is programmatically handled using React State.

Example: `<input type="text" />`